

Project 1 Midpoint Report

Stephen Byrd, Austin Devore

9/14/2026

Project Description

Part one of this project takes in an input string and parses it in a way that can be used for part two, which focuses more on processes, input/output redirection, and execution in the background/foreground.

Achievements

In part one, I implemented the ability to parse user input into different variables, enter debug mode, and exit when exit is entered. `getline()` is used to get user input, while `strtok()` is the key function used to parse the input into usable tokens. I also used `strcmp()` to compare strings when checking for debug mode and the exit command. The main runtime error I encountered was a segmentation fault when a command contained no normal arguments, because my program attempted to access the first argument even though none existed.

Preliminary Testing

Test Case 1: `myshell> one two <three >four &`

InputRedirect: [three]

OutputRedirect: [four]

Background: [1]

ArgumentCount: [2]

ArgumentVector[0]: [one]

ArgumentVector[1]: [two]

I believe the program passed this test because it correctly handles normal arguments, input/output redirection, and background execution. I chose this test because it is the standard example provided in the project description.

Test Case 2: `myshell> one two &`

InputRedirect: [NULL]

OutputRedirect: [NULL]

Background: [1]

ArgumentCount: [2]

ArgumentVector[0]: [one]

ArgumentVector[1]: [two]

I believe the program passed this test because the `&` correctly changes the Background variable to 1 while the other arguments remain in the argument vector.

Test Case 3: `myshell> one two < three > four`

Error: input filename required

I believe the program passed this test because I chose to require the filename to immediately follow the < or > symbol and return an error when a space is placed between them.

Next Steps

In Part II, I will build on the parser from Part I to actually execute the commands entered by the user. I will use `fork()` and `exec()` to create and run new processes, and `freopen()` to handle input and output redirection. I will also use `wait()` or `waitpid()` to manage foreground and background processes, handle invalid commands with an error message, and make sure all child processes have finished before the shell exits.